Assignment 6B

Aim

Write a program to implement insert, delete and traversal on a doubly linked list.

Theory

A doubly linked list is a type of linked list where each node contains part data, a pointer to the next node and a pointer to the previous node. This allows traversal in both forward and backward directions.
Insertion and deletion are easier compared to a singly linked list because we can directly access the previous node using the extra pointer. however it requires more memory since each node stores two links instead of one.

Insertion

To insert in a doubly linked list, a new node is created and its next and previous pointers are adjusted properly. The surrounding nodes links are also updated so that the new node is correctly connected from both sides.

Traversal

Traversal can be done from start to end by checking the next node and it is done from end to start by checking the

THADOMAL SHAHANI
TSEC
ENGINEERING COLLEGE

previous node.

Time complexity

O(n) for ~~insertion~~ ~~and~~ deletion and insertion ~~O(n) for deletion~~ as the linked list is sorted and it iterates until the correct position.

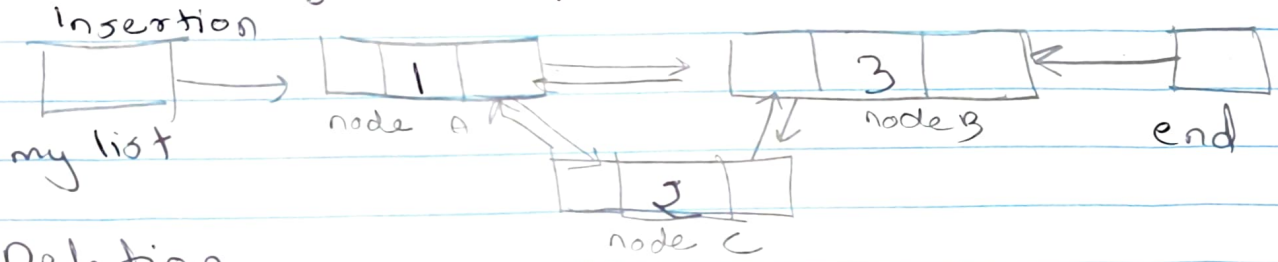
O(n) - Traversal as it iterates over the entire linked list.

Conclusion.

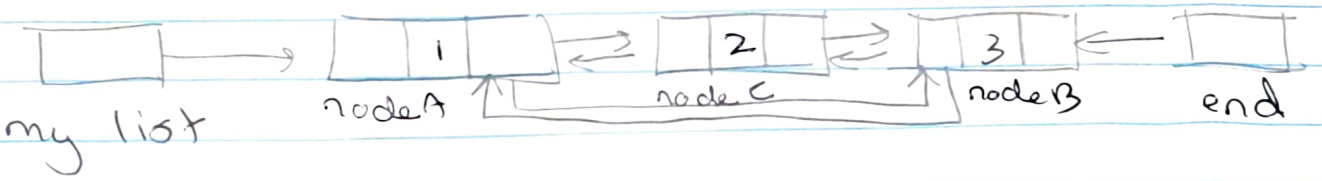
The concept is applied to evaluate problems in a variety of applications.

Insertion

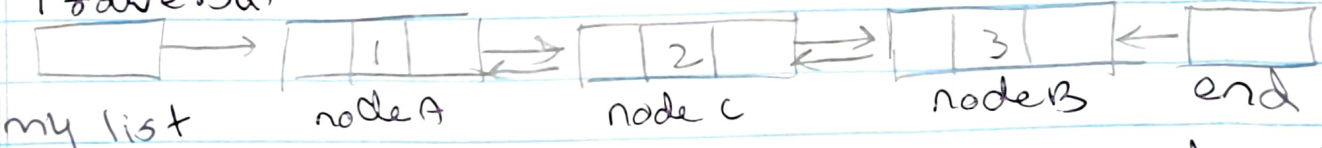
e.g.



Deletion



Traversal



It can be done from start to end and end to start

```
#include <stdio.h>
#include <stdlib.h>

struct ll_node {
   int data;
   struct ll_node *next;
   struct ll_node *prev;
};

struct ll_node* insert(struct ll_node *head, int val);
struct ll_node* del(struct ll_node* head, int val);
void display(struct ll_node *head);

void main() {

   struct ll_node *myList = NULL;
   int choice, value;

   do {
      printf("\n------ MENU ------\n");
      printf("1-Add Element\n");
      printf("2-Remove Element\n");
      printf("3-Show Linked List\n");
      printf("4-Quit\n");
      printf("Select option: ");
      scanf("%d", &choice);

      switch(choice) {
```

```
        case 1:
            printf("Input number to insert: ");
            scanf("%d", &value);
            myList = insert(myList, value);
            break;
        case 2:
            printf("Input number to remove: ");
            scanf("%d", &value);
            myList = del(myList, value);
            break;

        case 3:
            display(myList);
            break;

        case 4:
            printf("Program terminated.\n");
            break;

        default:
            printf("Wrong choice! Try again.\n");
    }

  } while(choice != 4);
}

struct ll_node* del(struct ll_node* head, int val) {
```

```
if (head == NULL) {
   printf("LIST IS EMPTY\n");
   return head;
}

struct ll_node *temp = head;

while (temp != NULL && temp->data != val)
   temp = temp->next;

if (temp == NULL) {
   printf("ELEMENT NOT FOUND\n");
   return head;
}
if (temp->prev == NULL) {
   head = temp->next;
   if (head != NULL)
      head->prev = NULL;
}
else {
   temp->prev->next = temp->next;

   if (temp->next != NULL)
      temp->next->prev = temp->prev;
}

free(temp);
```

```
    return head;
}

struct ll_node* insert(struct ll_node *head, int val) {

    struct ll_node *newNode = malloc(sizeof(struct ll_node));
    newNode->data = val;
    newNode->next = NULL;
    newNode->prev = NULL;
    if (head == NULL)
        return newNode;
    if (val < head->data) {
        newNode->next = head;
        head->prev = newNode;
        return newNode;
    }

    struct ll_node *temp = head;
    while (temp->next != NULL && temp->next->data < val)
        temp = temp->next;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
```

```
    return head;
}

void display(struct ll_node *head) {

    printf("\nLinked List Elements:\n");

    while (head != NULL) {
        printf("%d  ", head->data);
        head = head->next;
    }

    printf("\n");
}
```

```
------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 1
Input number to insert: 10

------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 1
Input number to insert: 30

------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 1
Input number to insert: 20

------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 3

Linked List Elements:
10  20  30
```

```
------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 2
Input number to remove: 10

------ MENU ------
1-Add Element
2-Remove Element
3-Show Linked List
4-Quit
Select option: 3

Linked List Elements:
20  30
```